

Object Oriented Software Engineering

Dr. Madhavi Dave
Project Manager
IoT Security
DRDO Industry
Academia SVP CoE

Unit 1: Introduction to Software Engineering and Modeling through UML

Building AI is Easy.

Building Reliable AI Products is Software
Engineering.

Causes of the Failure of AI Models

The AI model may be excellent, but failures can happen because of:

- Poor software architecture
- Database failures
- API issues
- Security vulnerabilities
- Concurrency problems
- Deployment mistakes

AI is only one component of a software system.

Real World AI Products

AI Product	SE Components
ChatGPT	APIs, databases, authentication, UI, cloud deployment
Netflix	Recommendation engine + backend services
Google Maps	AI + distributed systems + databases
Tesla	AI + embedded software + real-time systems
Amazon	AI + microservices + payment systems

Introduction to Software Engineering and UML

- Software Engineering

- Software engineering applies disciplined engineering principles to manage complexity, improve quality, and reduce project risk.
- It organizes work into distinct phases—requirements, design, coding, testing, deployment, and maintenance—so teams plan, verify, and adapt.

- Object Oriented Concepts

- UML

- Software Process Models

Why Software Engineering?

- Software development has evolved from writing simple programs to building complex enterprise systems involving millions of lines of code.
- Managing such complexity requires systematic engineering principles rather than ad hoc programming practices.

Problems without Software Engineering

- Late delivery
- Budget overruns
- Poor quality
- Difficult maintenance
- Security vulnerabilities

What is Software Engineering?

Software Engineering is the application of engineering principles to the design, development, testing, deployment, and maintenance of software.

Key Characteristics

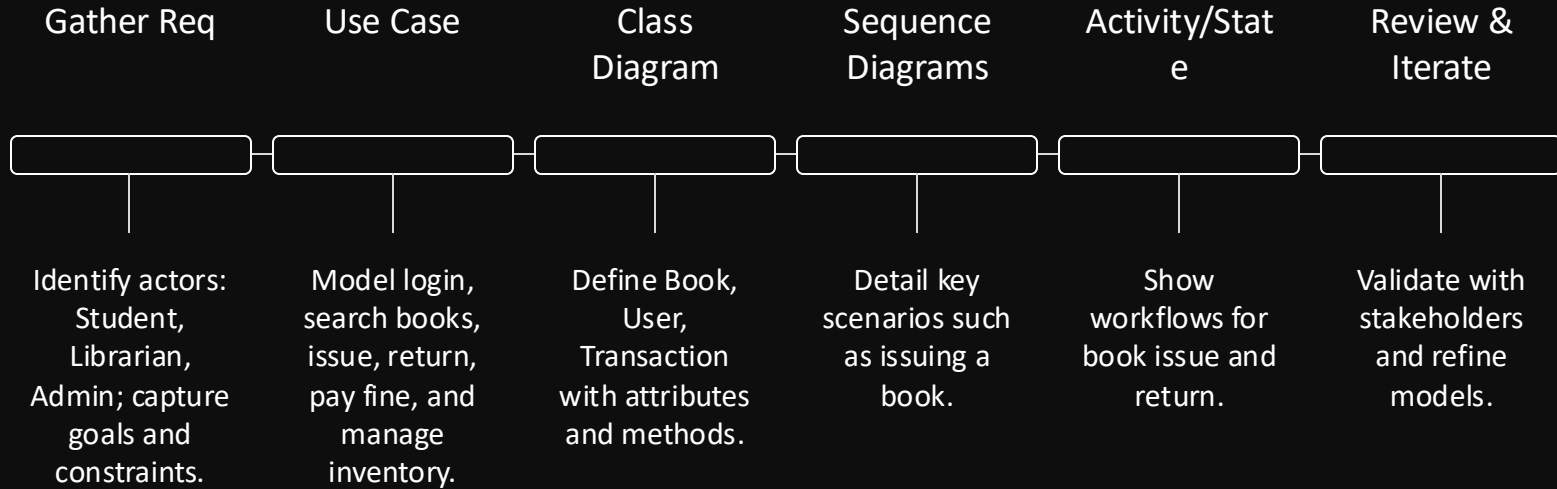
- Systematic
- Disciplined
- Quantifiable
- Quality focused
- Customer oriented

Phases of Software Engineering

ing.

- Requirements
- Design
- Coding
- Testing
- Deployment
- Maintenance

Modeling Timeline



Match to Context

Prescriptive Models

Waterfall is best when requirements are stable and changes are rare. The V-Model centers testing at every stage. Incremental and Spiral introduce controlled flexibility for larger or riskier efforts; Prototyping helps clarify unclear needs.

Specialized Models

Specialized models target specific goals: Component-Based Development favors reuse; Concurrent Development aligns parallel work; Formal Methods assure correctness in safety-critical domains. Agile complements them for rapid change. Choose based on domain: Agile for change, Formal Methods for safety,

Different projects require different process models—choose based on requirements stability, risk, and need for iteration.

Software Crisis

Major Problems

- Cost exceeded estimates
- Delayed delivery
- Low reliability
- Difficult maintenance
- Frequent failures

Example

Mars Climate Orbiter failure

Reason:

Mismatch between Metric and Imperial units.

Evolution of Programming

- Machine Language
- Assembly Language
- Procedural Programming
- Structured Programming
- Object Oriented Programming
- Object Oriented Software Engineering
- Component Based Systems
- Cloud & AI Software Engineering

Object Oriented Software Engineering

OOSE combines

- Software Engineering
- Object Oriented Programming
- Modeling

to develop reusable, maintainable and
scalable software systems.

Objects



Classes



UML



Software Process



Final Product

Object Oriented Concepts

- Object
- Class
- Attribute
- Method
- Message Passing
- Abstraction
- Encapsulation
- Inheritance
- Polymorphism

What is Modeling?

Modeling is the process of representing a real-world system using diagrams before implementation.

Benefits

- Better communication
- Early error detection
- Easier maintenance
- Documentation

Why UML?

Unified Modeling Language

Standard visual language

Used for

- Analysis
- Design
- Documentation
- Communication

UML Categories

- Structural
- Behavioral
- Interaction

UML Snapshot

Key UML artifacts and counts to remember for course deliverables.

3

UML diagram categories: Structural, Behavioral, Interaction.

5

Structural types: Class, Object, Component, Deployment, Package.

3

Behavioral types: Use Case, Activity, State Machine.

UML Categories

Structural

- Class
- Object
- Component
- Deployment
- Package

Behavioral

- Use Case
- Activity
- State Machine

Interaction

- Sequence
- Communication
- Timing